

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчет по лабораторной работе № 2
по дисциплине
«Объектно-ориентированное программирование»

Выполнил студент гр. ИВТб-2301-05-00

_____ /Елпашев С.А./
(подпись)

Проверил преподаватель

_____ /Шмакова Н.А./
(подпись)

1 Цель работы

Цель работы: изучить и освоить принципы объектно-ориентированного программирования, включая инкапсуляцию, наследование и полиморфизм, путем создания иерархии классов в выбранной предметной области, взаимодействие с базой данных и графическим интерфейсом.

2 Задание

На основании согласованной в первой лабораторной работе предметной области разработать приложение с графическим интерфейсом пользователя, используя классы и особенности C#. Сведения об объектах предметной области необходимо хранить в БД.

3 Решение

Проект реализован как ASP.NET Core Web API для управления парком ЭВМ. Предметная область разделена на несколько слоев: модели описывают персональные компьютеры и серверы, контроллер предоставляет HTTP-эндпоинты, сервис содержит бизнес-логику, а контракты и ответы формируют структуру запросов и ответов API.

Подключение к базе данных реализовано через Entity Framework Core. Основным классом доступа к данным — `AppDbContext` из файла `Data/AppDbContext.cs`. Он наследуется от `DbContext` и получает в конструкторе объект `DbContextOptions<AppDbContext>`, который передается в базовый класс. Entity Framework Core выступает в роли ORM, связывающей объекты предметной области с таблицами базы данных, а также обеспечивает выполнение запросов и сохранение изменений в PostgreSQL.

Добавление контекста производится в `Program.cs` через метод `AddDbContext`. В качестве провайдера используется PostgreSQL. Строка подключения берется из конфигурации приложения по ключу

`ConnectionStrings:Postgres` через

`builder.Configuration.GetConnectionString("Postgres")`. После запуска приложения контекст извлекается из области видимости зависимостей, и для базы данных вызывается `Database.EnsureCreated()`, чтобы при отсутствии БД автоматически создать схему.

В приложении реализована CRUD-модель работы с данными. Для объектов `Server` и `PC` предусмотрены следующие операции:

- **Create** — создание новой записи. Для серверов это метод `CreateServer`, который принимает данные из запроса, формирует объект `Server` и передает его в сервисный слой для добавления в базу данных. Для компьютеров используется аналогичный сценарий через метод создания объекта `PC`.

- **Read** — чтение данных. Метод `GetAllServers` возвращает список всех серверов, сохраненных в системе, а дополнительные методы получения информации возвращают сведения по выбранному объекту. Для компьютеров чтение реализовано по тому же принципу.

- **Update** — обновление существующей записи. Метод `UpdateServer` находит сервер по идентификатору, изменяет его характеристики и сохраняет новые значения в базе данных. Для класса `PC` обновление выполняется аналогичным образом.

- **Delete** — удаление записи. Метод `DeleteServer` удаляет сервер по уникальному идентификатору. Для объектов `PC` удаление выполняется по той же схеме через сервисный слой.

Сервисный слой `ComputerLabService` использует `AppDbContext` для добавления, изменения, удаления и выборки сущностей, а затем сохраняет изменения в базе данных.

Базовый абстрактный класс «`Computer`» содержит:

- Свойства:

- `public int Id { get; set; }` – свойство, означающее уникальный идентификатор;
- `public int ProcessorFrequency { get; set; }` – свойство, означающее частоту процессора в мегагерцах;
- `public int RamAmount { get; set; }` – свойство, означающее объем оперативной памяти в мегабайтах;
- Конструкторы:
 - `protected Computer()` – защищенный конструктор без параметров;
 - `protected Computer(int processorFrequency)` – защищенный конструктор, принимает частоту процессора и задает объем ОЗУ по умолчанию равным 1024 МБ;
 - `protected Computer(int processorFrequency, int ramAmount)` – защищенный конструктор, принимает частоту процессора и объем оперативной памяти;
- Абстрактные методы:
 - `public abstract string DisplayInfo()` – публичный абстрактный метод, возвращающий информацию об объекте;
 - `public abstract string ExecuteTask()` – публичный абстрактный метод, возвращающий описание выполняемой задачи.

Класс «РС» содержит:

- Свойства:
 - `public string UserShell { get; set; }` – свойство, означающее пользовательскую оболочку;
 - `public string Os { get; set; }` – свойство, означающее операционную систему;
- Конструкторы:

- `public PC() : base()` – публичный конструктор без параметров, задает значения по умолчанию;
- `public PC(int processorFrequency, int ramAmount) :`
`base(processorFrequency, ramAmount)` – публичный конструктор, принимает частоту процессора и объем ОЗУ;
- `public PC(int processorFrequency, int ramAmount, string userShell) :`
`base(processorFrequency, ramAmount)` – публичный конструктор, принимает частоту процессора, объем ОЗУ и пользовательскую оболочку;
- `public PC(int processorFrequency, int ramAmount, string userShell, string os) :`
`base(processorFrequency, ramAmount)` – публичный конструктор, принимает частоту процессора, объем ОЗУ, пользовательскую оболочку и операционную систему;
- Методы:
 - `public void IncreaseRamAmount(int amount)` – публичный метод, увеличивающий объем ОЗУ на заданное значение;
 - `public override string DisplayInfo()` – переопределенный метод, возвращающий информацию о персональном компьютере;
 - `public override string ExecuteTask()` – переопределенный метод, возвращающий описание выполняемой задачи;
 - `public string OpenApplication(string appName)` – публичный метод, возвращающий строку с сообщением об открытии приложения;
 - `public string OpenWebsite(string url)` – публичный метод, возвращающий строку с сообщением об открытии сайта.

Класс «Server» содержит:

- Свойства:
 - `public int MaxConnections { get; set; }` – свойство, означающее максимальное количество подключений;

- `public int CurrentConnections { get; private set; }` – свойство, означающее текущее количество подключений;
- Конструкторы:
 - `public Server() : base()` – публичный конструктор без параметров, задает значения по умолчанию;
 - `public Server(int processorFrequency, int ramAmount, int maxConnections) :`
`base(processorFrequency, ramAmount)` – публичный конструктор, принимает частоту процессора, объем ОЗУ и максимальное количество подключений;
 - `public Server(int processorFrequency, int ramAmount, int maxConnections, int currentConnections) :`
`base(processorFrequency, ramAmount)` – публичный конструктор, принимает частоту процессора, объем ОЗУ, максимальное и текущее количество подключений;
- Методы:
 - `public void Update(int processorFrequency, int ramAmount, int maxConnections, int currentConnections)` – публичный метод, обновляющий значения основных характеристик сервера;
 - `public void IncrementConnection()` – публичный метод, увеличивающий число активных подключений на единицу;
 - `public override string DisplayInfo()` – переопределенный метод, возвращающий информацию о сервере;
 - `public override string ExecuteTask()` – переопределенный метод, возвращающий описание выполняемой задачи;
 - `public bool AcceptConnection(out string message)` – публичный метод, пытающийся принять новое подключение и возвращающий сообщение о результате;

– `public string ProcessRequest()` – публичный метод, возвращающий строку с сообщением об обработке сетевого запроса.

Далее идёт создание контроллера для ответов на запросы для клиентов. Методы для работы с серверами в контроллере реализованы следующим образом:

– `public async Task<ActionResult<IEnumerable<ServerSummaryResponse>> GetAllServers(CancellationToken cancellationToken)` – метод, возвращающий список всех серверов, сохраненных в системе;

– `public async Task<ActionResult<CreatedResourceResponse> CreateServer([FromBody] CreateServerRequest request, CancellationToken cancellationToken)` – метод, создающий новый сервер, использующий проверку входных данных и метод `AddServerAsync` для добавления записи в базу данных:

```
var server = new Server(request.ProcessorFrequency,
    request.RamAmount, request.MaxConnections,
    request.CurrentConnections);
var id = await _computerLabService.AddServerAsync(server,
    cancellationToken);
```

– `public async Task<ActionResult<ServerSummaryResponse> UpdateServer([FromRoute] int id, [FromBody] CreateServerRequest request, CancellationToken cancellationToken)` – метод, обновляющий существующий сервер по уникальному идентификатору;

– `public async Task<IActionResult> DeleteServer([FromRoute] int id, CancellationToken cancellationToken)` – метод, удаляющий сервер по уникальному идентификатору;

– `public async Task<ActionResult<OperationMessageResponse> GetServerInfo([FromRoute] int id, CancellationToken cancellationToken)` – метод, возвращающий полное описание сервера по уникальному идентификатору, используя метод `DisplayInfo()`;

4 Скриншоты работы программы

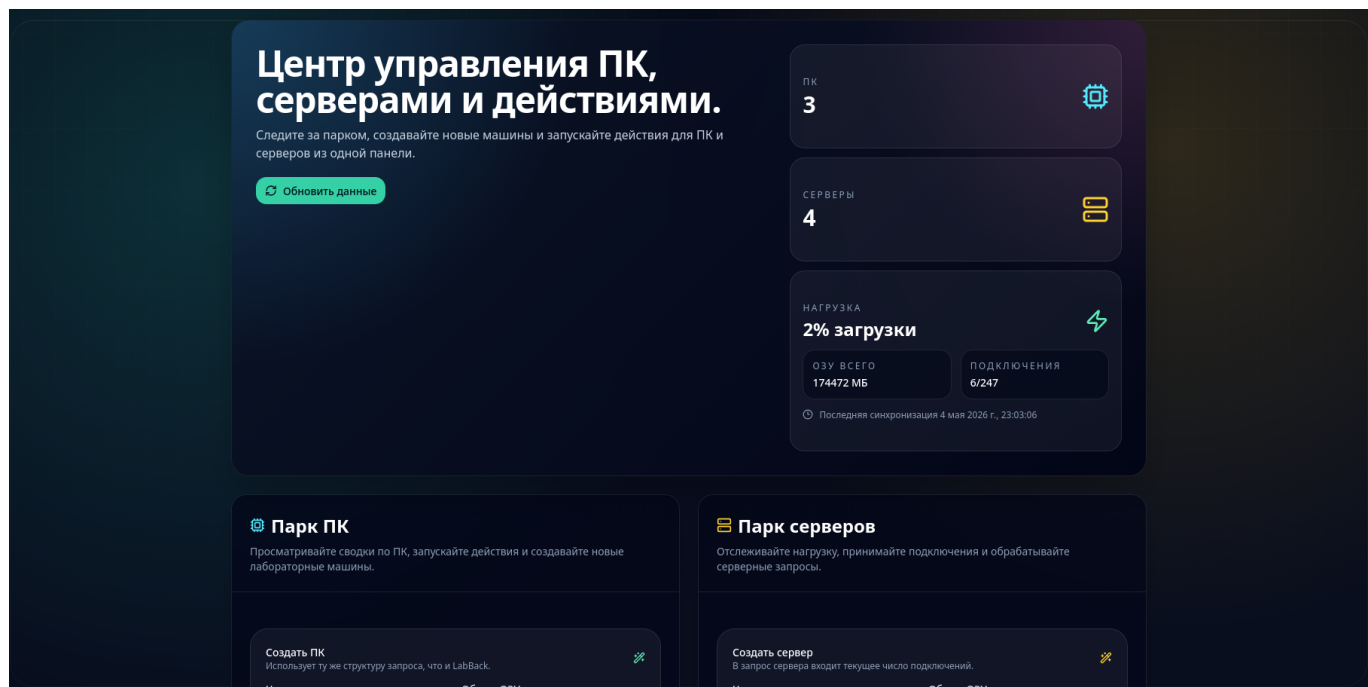


Рис. 2: Главный экран сайта

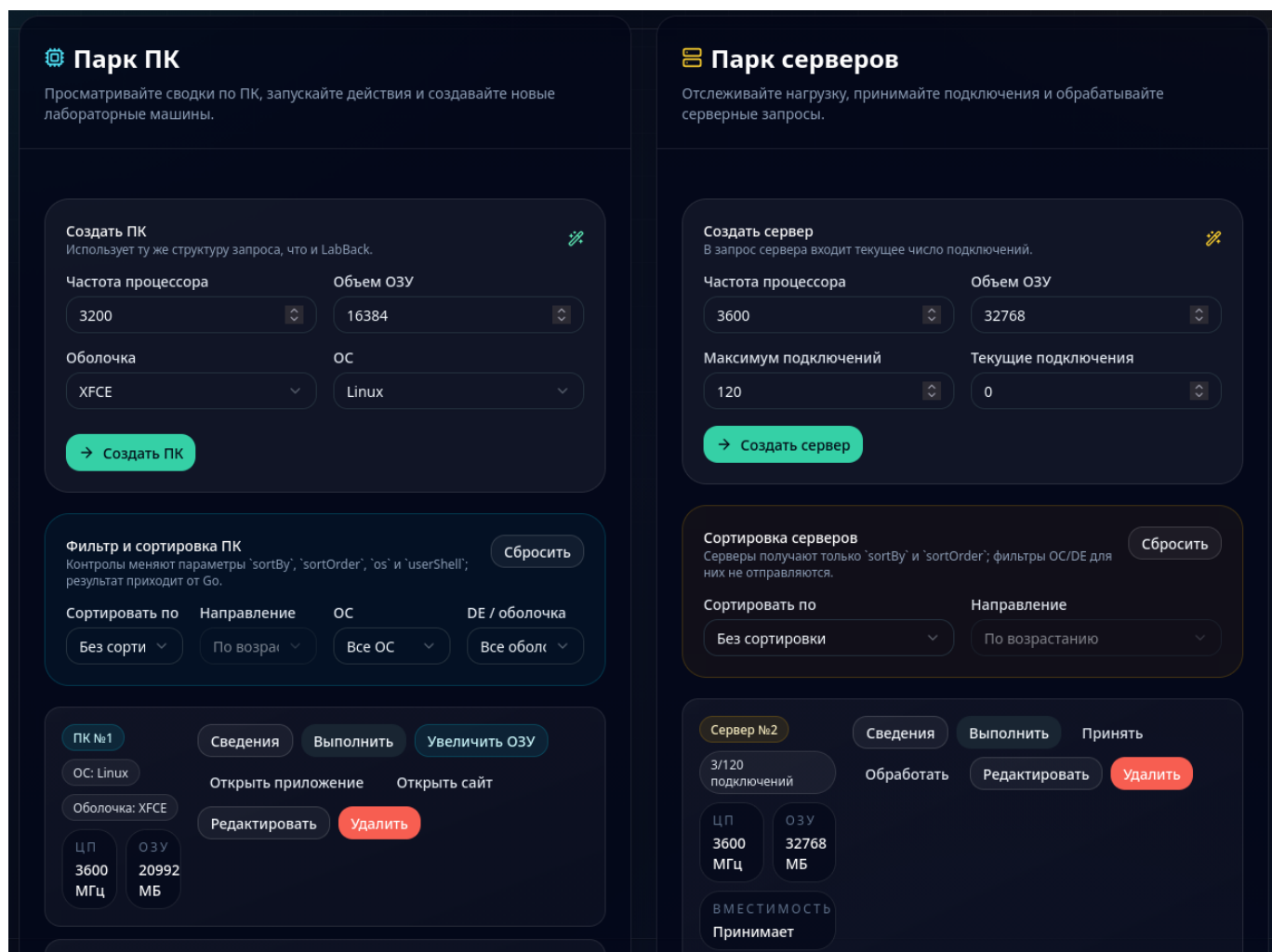


Рис. 3: Список серверов и компьютеров

Создать ПК

Использует ту же структуру запроса, что и LabBack.

Частота процессора

3200

Объем ОЗУ

16384

Оболочка

GNOME

ОС

Linux

→ Создать ПК

Создать ПК

ПК создан

23:04:25

Рис. 4: Вывод сообщения о добавлении

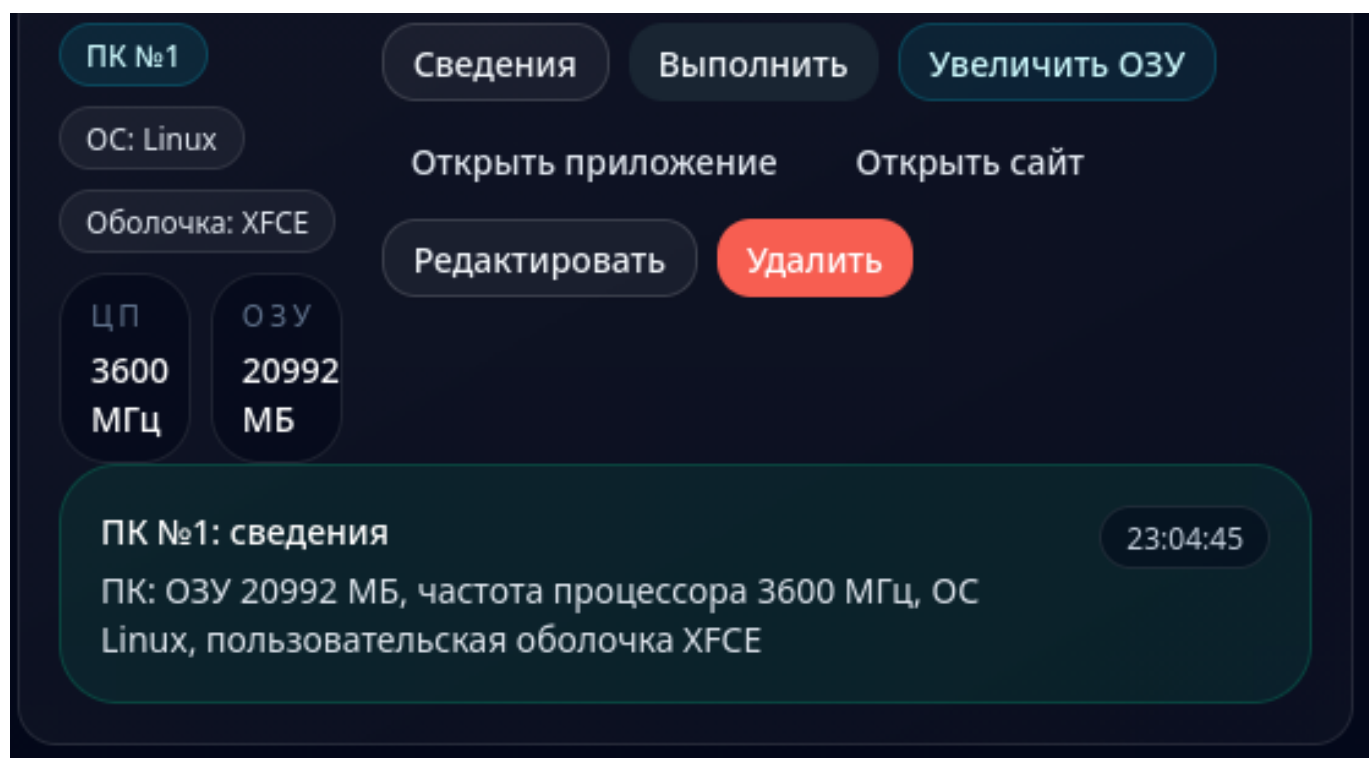


Рис. 5: Вывод информации об объекте

Исходный код выполненной лабораторной работы представлен в текущем репозитории проекта – .

5 Вывод

В ходе выполнения лабораторной работы была разработана и реализована иерархия классов и сопутствующая инфраструктура Web API для управления парком ЭВМ. Изучены основные принципы объектно-ориентированного программирования: инкапсуляция, наследование и полиморфизм. Разработано приложение, демонстрирующее применение данных принципов на уровне доменной модели, контроллера и слоя доступа к данным.